

-
2. This question involves the `SignedText` class, which contains methods that are used to include a signature as part of a string of text. You will write the complete `SignedText` class, which contains a constructor and two methods.

The `SignedText` constructor takes two `String` parameters. The first parameter is a first name and the second parameter is a last name. The length of the second parameter is always greater than or equal to 1.

The `getSignature` method takes no parameters and returns a formatted signature string constructed from the first and last names according to the following rules.

- If the first name is an empty string, the returned signature string contains just the last name.
- If the first name is not an empty string, the returned signature string is the first letter of the first name, a dash ("–"), and the last name, in that order.

The `addSignature` method returns a possibly revised copy of its `String` parameter. The parameter will contain at most one occurrence of the object's signature, at either the beginning or the end of the parameter. The returned string is created from the parameter according to the following rules.

- If the object's signature does not occur in the `String` parameter of the method, the returned `String` is the value of the parameter with the signature added to the end.
- If the object's signature occurs at the end of the `String` parameter, the returned `String` is the unchanged value of the parameter.
- If the object's signature occurs at the beginning of the `String` parameter, the returned `String` is the value of the original parameter with the signature removed from the beginning and appended to the end of the parameter.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than `SignedText`.

Statement	Method Call Return Value (blank if none)	Explanation
<code>SignedText st1 = new SignedText("", "Wong");</code>		The <code>SignedText</code> object <code>st1</code> has an empty first name and last name "Wong".
<code>String temp = st1.getSignature();</code>	"Wong"	
<code>SignedText st2 = new SignedText("henri", "dubois");</code>		The <code>SignedText</code> object <code>st2</code> has first name "henri" and last name "dubois".
<code>temp = st2.getSignature();</code>	"h-dubois"	
<code>SignedText st3 = new SignedText("GRACE", "LOPEZ");</code>		The <code>SignedText</code> object <code>st3</code> has first name "GRACE" and last name "LOPEZ".
<code>temp = st3.getSignature();</code>	"G-LOPEZ"	
<code>SignedText st4 = new SignedText("", "FOX");</code>		The <code>SignedText</code> object <code>st4</code> has an empty first name and last name "FOX".
<code>String text = "Dear";</code>		
<code>temp = st4.addSignature(text);</code>	"DearFOX"	The signature does not occur in the <code>addSignature</code> parameter, so the returned string is the value of the parameter with the signature appended.
<code>text = "Best wishesFOX";</code>		
<code>temp = st4.addSignature(text);</code>	"Best wishesFOX"	The signature occurs at the end of the <code>addSignature</code> parameter, so the returned string is the unchanged value of the parameter.

Statement	Method Call Return Value (blank if none)	Explanation
<code>text = "FOXThanks";</code>		
<code>temp = st4.addSignature(text);</code>	"ThanksFOX"	The signature occurs at the beginning of the <code>addSignature</code> parameter, so the returned string is the value of the original parameter with the signature removed from the beginning and appended to the end of the parameter.
<code>text = "G-LOPEZHello";</code>		
<code>temp = st3.addSignature(text);</code>	"HelloG-LOPEZ"	The signature occurs at the beginning of the <code>addSignature</code> parameter, so the returned string is the value of the original parameter with the signature removed from the beginning and appended to the end of the parameter.

Write the complete `SignedText` class. Your implementation must meet all specifications and conform to the examples in the table.